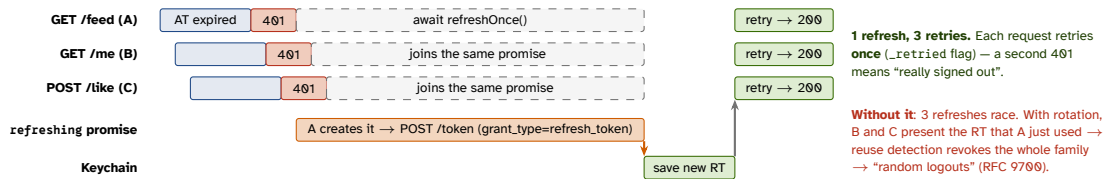


In one line: The protocol is the native one — **authorization code + PKCE in the system browser** (ASWebAuthenticationSession / Custom Tabs). The RN work is: a library that does it right, **refresh token in Keychain/Keystore**, access token **in memory**, **one refresh at a time** for every caller, and a **logout that wipes** every cache the user's data reached.

Single-flight refresh — three screens hit 401 at once



Better still: refresh before sending when `expiresAt - now < 60 s` (compute `expiresAt` from `expires_in` at receipt — not device clock vs exp) and on AppState 'active'. The 401 path is the safety net.

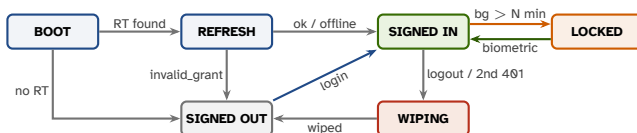
Signing in — the libraries

- expo-auth-session:** useAutoDiscovery(issuer), useAuthRequest (PKCE on by default, S256), promptAsync() opens expo-web-browser openAuthSessionAsync → ASWebAuthenticationSession / Custom Tabs; then exchangeCodeAsync, refreshAsync; makeRedirectUri({ scheme }). Expo Go redirects to exp: // ... — test real redirects in a dev build.
- react-native-app-auth:** wraps AppAuth-iOS / AppAuth-Android; authorize(cfg), refresh(cfg, { refreshToken }), revoke(cfg, { tokenToRevoke }); Android manifestPlaceholders = [appAuthRedirectScheme: 'com.example.app']; iosPrefersEphemeralSession.
- Never a WebView login:** the app can read the password, no SSO cookie; RFC 8252: apps MUST NOT use embedded user-agents, servers MAY block them. Redirect: claimed https (universal/app link) > custom scheme — any app can register a scheme; PKCE makes a stolen code useless.

Where tokens live — and the boot sequence

- RT:** Keychain/Keystore (react-native-keychain, expo-secure-store). **AT:** a module variable / non-persisted store. Never redux-persist, AsyncStorage, MMKV-plain, logs or crash breadcrumbs.
- Boot:** keep the splash (SplashScreen.preventAutoHideAsync()) → read RT → refresh → pick the navigator. No RT → auth stack. Offline → stay signed in with cached data; only invalid_grant means signed out.
- Reinstall:** iOS Keychain outlives the app, AsyncStorage does not — no “installed” flag in AsyncStorage ⇒ wipe Keychain items first.
- Rotation:** persist the new RT before anything else; two processes (app + background task) must not both refresh — share the lock or skip in background.

Session states



Biometric re-auth

expo-local-authentication authenticateAsync() returns a **JS boolean** — a UI gate, hookable on a rooted phone. Stronger: gate the secret — a Keychain item with ACCESS_CONTROL.BIOMETRY_CURRENT_SET: the OS releases the RT only after Face ID; a new enrolment invalidates it (iOS: NSFaceIDUsageDescription). Payments, password change: server-side step-up (max_age / prompt=login).

Example — axios interceptor, single-flight refresh

```
let refreshing: Promise<string> | null = null;
function refreshOnce(): Promise<string> {
  refreshing ??= doRefresh() // the first caller creates it
  .finally(() => { refreshing = null; }); // next expiry: new one
  return refreshing; // everyone else joins it
}
api.interceptors.response.use(
  undefined,
  async (err) => {
    const cfg = err.config;
    if (err.response?.status !== 401 || cfg._retried) throw err;
    cfg._retried = true; // retry ONCE, never loop
    try {
      const at = await refreshOnce();
      cfg.headers.Authorization = `Bearer ${at}`;
      return api(cfg); // replay the original call
    } catch (e) {
      if (isInvalidGrant(e)) await signOut('expired'); // not on offline
      throw e;
    }
  }
);
// doRefresh(): read RT from Keychain, POST /token with a BARE
// client (no interceptor -> no recursion), save new RT, return AT
```

Logout — wipe everything the user reached

- revoke the RT at the server (RFC 7009) — best effort; offline still logs out.
- Delete secure items: Keychain.resetGenericPassword() / SecureStore.deleteItemAsync(); clear the in-memory AT; abort in-flight requests (AbortController).
- Caches: queryClient.clear(), reset the Redux root / Zustand store, MMKV clearAll(), image + file caches, drafts.
- Unregister the push token server-side; Sentry.setUser(null); analytics reset.
- Navigation: auth-conditional screens swap stacks by themselves (else navigation.reset) — no “back” into the app.
- Browser session: end_session_endpoint, or the next login is silent SSO.

Traps

- Logging out on any refresh failure — airplane mode signs everyone out.
- Refresh through the intercepted client → 401 inside refresh → infinite loop.
- Logout that forgets the query cache: user B sees user A's feed.
- Biometric check as a boolean in front of an RT stored without access control.

Remember

Browser + PKCE · RT in Keychain, AT in memory · one refresh, one retry · wipe it all.